

Statement of Work

Information System Module

Interactive System for Macaw Welfare

Planète Sauvage - Port-Saint-Père



Full Stack Web Interface

Angular Frontend + Spring Backend

Macaw Project Team

Supervisors: Thomas Lechevallier and Jean-Yves Martin

Participants: BOUNOUA Ilyas and VAZEILLE Clément

October 2025

Abstract

This document constitutes the simplified statement of work for the Information System module of the Macaw project. This version defines the functional and technical specifications for our web interface, enabling basic supervision and data consultation of the interactive system designed for macaws at Planète Sauvage.

Contents

1	Context and Objectives of the IS Module	3
1.1	Project context recap	3
1.2	Module objectives	3
1.3	Non-objectives (out of scope)	3
2	Information System Architecture	3
2.1	Architectural overview	3
2.2	Technology stack	4
2.3	Backend software architecture	4
2.4	Frontend software architecture	5
3	Functional Specifications	5
3.1	Actors	5
3.2	System use case diagram	5
3.3	Essential functionalities	6
3.3.1	F1: Simple authentication	6
3.3.2	F2: Supervision dashboard	6
3.3.3	F3: Interactions list	6
3.3.4	F4: Module details	6
3.3.5	F5: Basic module configuration	7
3.3.6	F6: Simple CSV export	7
3.4	User interfaces	7
3.4.1	Main pages	7
3.4.2	Navigation	8
4	Backend Technical Specifications	8
4.1	Simplified REST API	8
4.1.1	Essential endpoints	8
4.2	Physical Data Model	9
5	Frontend Technical Specifications	9
5.1	Main components	9
5.1.1	Dashboard Component	9
5.1.2	InteractionsComponent	9

5.1.3	ModuleDetailComponent	10
5.2	Simplified UI/UX	10
6	Schedule and Distribution	10
6.1	50-hour distribution	11
6.2	8-week schedule	11
7	Testing	11
7.1	Backend testing	11
7.2	Frontend testing	12
8	Deployment	12
8.1	Backend	12
8.2	Frontend	12
9	Accepted limitations	12
10	Deliverables	13
10.1	Code	13
10.2	Documentation	13
10.3	Application	13
11	Validation criteria	13
11.1	Functional criteria	13
11.2	Technical criteria	14
12	Conclusion	14
12.1	Guiding principles	14
12.2	Success factors	14

1 Context and Objectives of the IS Module

1.1 Project context recap

The Macaw project aims to develop an interactive system to improve the welfare of captive macaws at Planète Sauvage. The objective of the IS module is to create a web interface that will enable supervision and data consultation of the system as well as remote system control. The interaction system with macaws consists of mechanisms that the macaw can activate with its beak or feet, producing sounds in response to interaction. The objective of the IS system is to analyze the system's usage by macaws and remotely configure it for optimization and macaw research purposes.

1.2 Module objectives

1. Provide a simple and functional web interface
2. Visualize macaw interaction data
3. Monitor Raspberry Pi module status
4. Enable basic remote configuration
5. Sound library management

1.3 Non-objectives (out of scope)

- Multiple user management
- Automatic reports and emails
- Advanced graphs
- Mobile application
- Real-time features

2 Information System Architecture

2.1 Architectural overview

Simplified three-tier architecture:

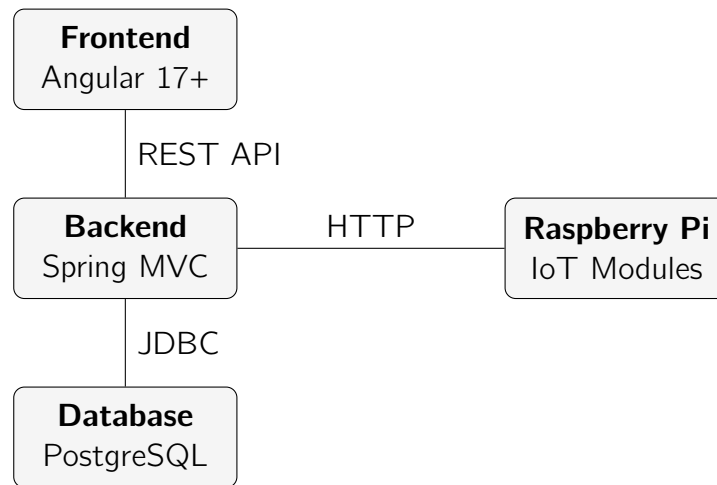


Figure 1: Simplified IS module architecture

2.2 Technology stack

Layer	Technology	Version
Frontend	Angular	17+
Frontend Language	TypeScript	5.x
UI Framework	Angular Material	17+
Backend	Spring MVC	5.3+
Backend Language	Java	17 LTS
Database	PostgreSQL	15+
Data Access	Spring JDBC Template	5.3+
API Communication	REST (JSON)	-
Frontend Build	npm / Angular CLI	-
Backend Build	Maven	3.9+
Application Server	Tomcat	9.x

Table 1: Technology stack

2.3 Backend software architecture

Simplified 3-layer architecture:

- **Controller Layer:** REST API endpoints
- **Service Layer:** Basic business logic
- **DAO Layer:** Data access with JDBC Template

2.4 Frontend software architecture

Simplified Angular structure:

- **Services:** HTTP API communication
- **Models:** TypeScript interfaces
- **Routing:** Simple navigation

3 Functional Specifications

3.1 Actors

Actor	Description
Administrator	Single user with full access (hardcoded login)
System	Raspberry Pi modules sending and receiving data

Table 2: System actors

3.2 System use case diagram

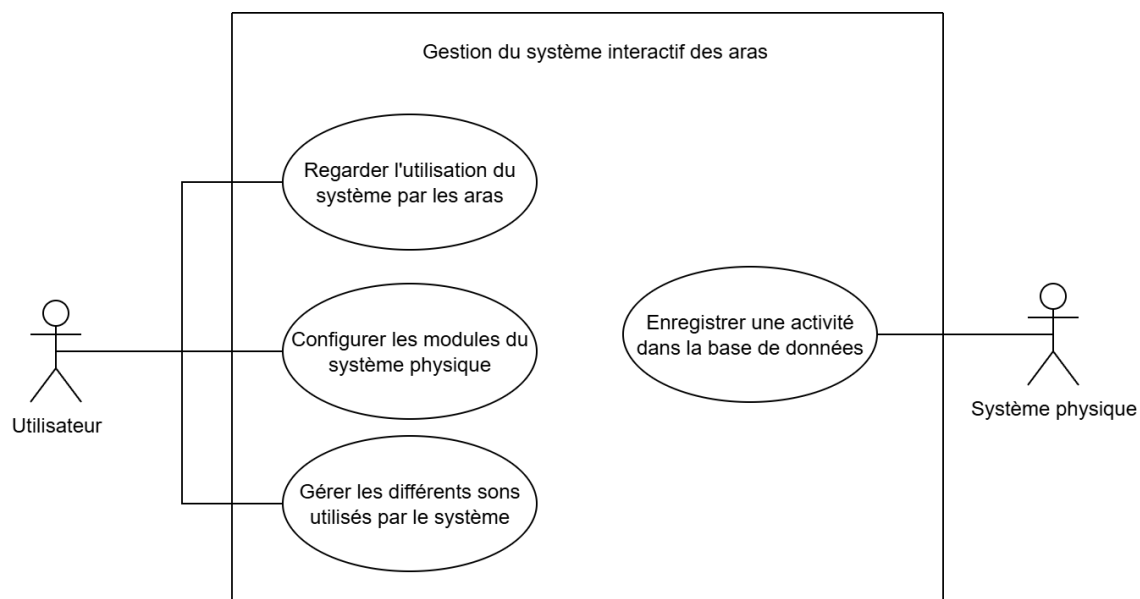


Figure 2: System use case diagram

3.3 Essential functionalities

3.3.1 F1: Simple authentication

- Login page with username/password
- Credentials hardcoded in configuration
- Simple session (no JWT, classic HTTP session)
- Redirect to dashboard after login

Estimated time: 3 hours

3.3.2 F2: Supervision dashboard

- Display total daily interactions count
- Module list with status (active/inactive)
- Last recorded interaction
- Bar chart of usage throughout the day, week, and month
- Manual refresh (button)

Estimated time: 6 hours

3.3.3 F3: Interactions list

- Paginated interaction table (20 per page)
- Columns: Date/Time, Module, Type
- Sort by descending date
- Simple date filter (today/yesterday/week)

Estimated time: 8 hours

3.3.4 F4: Module details

- Detailed view of selected module
- Information: Name, Location, Status, IP
- Current configuration: Volume, Mode
- Activation/deactivation button

Estimated time: 5 hours

3.3.5 F5: Basic module configuration

- Simple form with 3 fields:
 - Volume (slider 0-100)
 - Mode (Manual/Automatic)
 - Active (Yes/No)
 - Sound (Yes/No)
- Save button
- Confirmation message

Estimated time: 4 hours

3.3.6 F6: Simple CSV export

- "Export" button on interactions list page
- Export displayed data (with applied filters)
- Direct CSV download

Estimated time: 6 hours

3.4 User interfaces

3.4.1 Main pages

1. **/login**: Login page
2. **/dashboard**: Dashboard
3. **/interactions**: Interactions list
4. **/modules**: Modules list
5. **/modules/:id**: Module detail and configuration
6. **/sounds**: Sounds list

3.4.2 Navigation

Fixed side menu with 3 items:

- Dashboard
- Interactions
- Modules
- Sounds

4 Backend Technical Specifications

4.1 Simplified REST API

4.1.1 Essential endpoints

Authentication

POST	/api/auth/login	# Login
POST	/api/auth/logout	# Logout

Dashboard

GET	/api/dashboard/summary	# Daily statistics
-----	------------------------	--------------------

Modules

GET	/api/modules	# Module list
GET	/api/modules/{id}	# Module detail
PUT	/api/modules/{id}/config	# Update config

Interactions

GET	/api/interactions	# Paginated list
POST	/api/interactions	# Create (IoT)
GET	/api/interactions/export	# CSV export

4.2 Physical Data Model

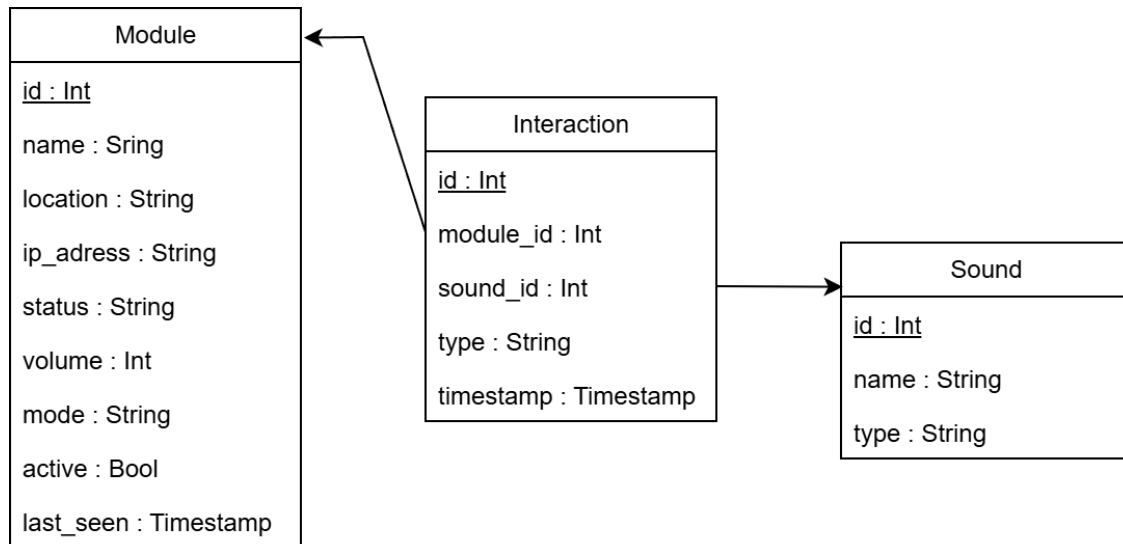


Figure 3: System Physical Data Model

5 Frontend Technical Specifications

5.1 Main components

5.1.1 Dashboard Component

- 3 Material cards: Total interactions, Active modules, Last interaction
- Simple list of last 5 interactions
- Bar chart of interactions
- Refresh button

5.1.2 InteractionsComponent

- Material table with pagination
- Simple filters (radio buttons)
- CSV export button

5.1.3 ModuleDetailComponent

- Angular reactive form
- Material slider for volume
- Select for mode
- Checkbox for active
- Save button

5.2 Simplified UI/UX

- Angular Material only (no custom CSS)
- Default Material palette
- Basic responsive layout
- No animations
- Simple error messages

6 Schedule and Distribution

This schedule only concerns the time allocated to product development, not the design phases or report writing.

6.1 50-hour distribution

Task	Backend	Frontend
Project setup (Maven, Angular CLI)	4h	3h
Spring configuration	3h	-
DB schema + SQL scripts	2h	-
Simple auth	1h	2h
Dashboard API	2h	4h
Modules API (CRUD)	3h	5h
Interactions API	3h	6h
CSV export	2h	2h
Basic unit tests	2h	2h
Deployment	2h	1h
Documentation	1h	1h
Total	25h	26h
Contingency	3h	
GRAND TOTAL	54h	

Table 3: Detailed time distribution

6.2 8-week schedule

Week	Deliverables	Hours
W1	Project setup + Environment configuration	7h
W2	Authentication + Database	6h
W3	Dashboard API + Dashboard interface	7h
W4	Modules API + Module list	7h
W5	Module configuration + Tests	7h
W6	Interactions API + Interactions list	7h
W7	CSV export + Final tests	6h
W8	Deployment + Documentation	4h
Total		51h
<i>Note: One week margin for contingencies or adjustments</i>		

Table 4: Weekly schedule over 8 weeks

7 Testing

7.1 Backend testing

Minimal unit tests with JUnit 5:

- Service method tests

7.2 Frontend testing

Minimal unit tests with Jasmine:

- Component creation tests
- Main service method tests

8 Deployment

8.1 Backend

- Maven build: `mvn clean package`
- Deploy on Tomcat 9
- External PostgreSQL configuration
- Port: 8080

8.2 Frontend

- Build: `ng build`
- Deployment: Copy dist/ to Tomcat webapps
- Or simple HTTP server (nginx)

9 Accepted limitations

- **Security:** Basic authentication, no JWT
- **Performance:** No advanced optimization
- **Scalability:** No cache, no optimized pool
- **UI/UX:** Functional but basic interface
- **Error handling:** Simple messages
- **Logging:** Console only

10 Deliverables

10.1 Code

1. Git repository with Backend and Frontend source code
2. SQL initialization scripts
3. README file with instructions

10.2 Documentation

1. This statement of work
2. Installation and usage guide
3. Simple API documentation (endpoint list)

10.3 Application

1. Deployable backend
2. Angular build
3. Database with test data

11 Validation criteria

11.1 Functional criteria

- ☐ Functional login
- ☐ Dashboard displays data
- ☐ Paginated interactions list
- ☐ Module configuration saved
- ☐ Downloadable CSV export

11.2 Technical criteria

- ☐ Project compiles without errors
- ☐ API responds in < 1s
- ☐ Responsive interface (desktop/tablet)
- ☐ Tests pass

12 Conclusion

This statement of work defines a realistic scope considering the time allocated to the project.

12.1 Guiding principles

- **Simplicity:** Essential functionalities only
- **Pragmatism:** Simple technical choices
- **Feasibility:** Realistic planning
- **Scalability:** Architecture allowing future extensions

12.2 Success factors

1. Strictly respect the scope
2. Do not add features
3. Prioritize functionality over aesthetics
4. Test as you go
5. Document important decisions

SOW
Macaw Project - Planète Sauvage
October 2025